

1 VLA-RL: 通过可扩展强化学习迈向高水平且通用的机器人操纵

摘要

近年来，高容量的 vision-language-action (VLA) 模型通过模仿人类 demonstration，在一系列机器人操纵任务上展现出令人瞩目的性能。然而，若仅利用已访问状态有限的离线数据，模型在分布外场景中往往会执行失败。直观来看，一种基于探索、并能够在测试时利用在线采集数据持续改进的方法，有望缓解这一局限。

本文提出 VLA-RL：一个利用在线 reinforcement learning (RL) 来提升预训练自回归 VLA 在下游任务中表现的算法与系统框架。在统一视角下，我们首先给出一种面向自回归 VLA 训练的轨迹级 RL 建模方式，将一般机器人操纵轨迹表示为多模态、多轮对话。针对奖励稀疏这一挑战，我们将预训练 vision-language model 微调为机器人过程奖励模型 (process reward model)；该模型基于自动抽取的任务片段，并使用其上标注的伪奖励标签进行训练。

为实现大规模训练，我们进一步总结了若干能够提升稳定性与效率的实现经验，包括 curriculum selection strategy、GPU-balanced vectorized environments、batch decoding，以及 critic warmup。实验表明，在 LIBERO 的 40 个具有挑战性的机器人操纵任务上，VLA-RL 使 OpenVLA-7B 的性能相比最强的 finetune baseline 提升了 4.5%，甚至能够匹配 π -FAST 等先进商业模型的表现。值得注意的是，我们观察到，随着测试时优化强度的增加，VLA-RL 还会持续受益，这表明机器人领域中可能已经初步显现出 inference scaling laws 的迹象。

“我们希望 AI agent 像人一样去发现，而不是仅仅包含我们已经发现的东西。” — The Bitter Lesson

2 Introduction

在互联网规模数据集上预训练的大型 foundation model，已经在文本 [56, 13, 17]、图像 [79, 74, 81, 80]、视频 [87, 73] 和音频 [14] 等多种领域展现出显著效果。近年来，研究者进一步基于大规模人类 demonstration [15, 20, 58, 52, 35, 23, 16, 10, 33, 18, 72, 6, 37] 提出了大型 vision-language-action (VLA) 模型 [9, 8, 68, 39, 44, 35, 36, 19, 6, 7, 57, 32]，这表明通向能够执行多样化操纵任务的通用机器人，可能存在一条可行路径。然而，仅依赖离线数据进行利用 (exploitation)，由于其访问到的状态有限，往往会在测试时的域外 (Out-of-Domain, OOD) 场景中导致执行失败。

我们认为，克服这一挑战的关键，在于将以利用为主的方法转变为以探索为主的方法，而强化学习 (RL) [63] 正是这一思路的代表。近期，RL 在大语言模型 (LLM) 上的应用取得了突破性进展 [17, 82, 47, 31]。这些结果表明，仅靠对离线网络数据进行模仿式预训练所带来的 scaling 收益，已经逐渐逼近上限。因此，通过在在线采集数据上训练，并借助几乎不受限的状态覆盖，强化学习正成为实现测试时 scaling 提升的一种有前景范式。一个自然的问题是：在机器人操纵领域，我们能否同样获得这种基于 RL 的测试时 scaling 收益？

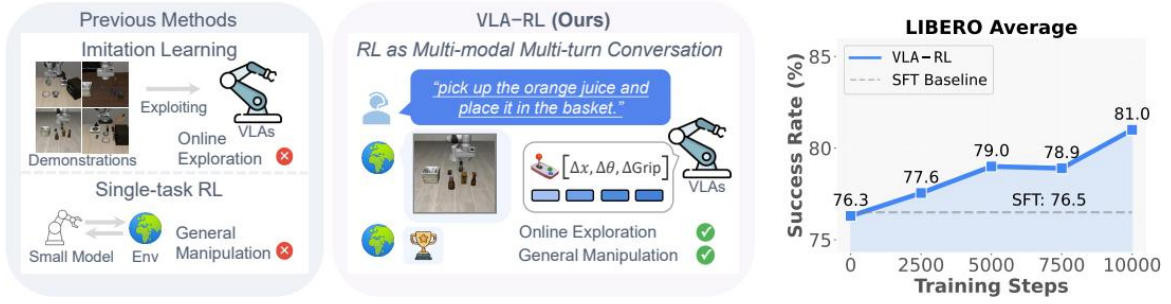


图 1: 以往的 VLA 主要关注利用离线 demonstrations 的模仿学习, 而 VLA-RL 则探索如何通过可扩展的强化学习进一步提升高容量 VLA。评测中, 我们训练 OpenVLA-7B 在 LIBERO 的 40 个高难度机器人操纵任务上取得能力提升, 并相较于模仿学习 baseline 展现出显著且一致的改进。

另一方面, 已有大量工作尝试将 RL 应用于机器人领域 [66]。然而, 传统从零开始训练的 RL 往往存在数据效率低、并且需要大量 reward engineering 的问题 [2, 29, 85, 91]。因此, 以往研究大多聚焦于较简单的场景, 例如低维状态空间 [60, 24, 54]、小规模网络架构 (如 MLP) [60, 70, 1, 50, 51], 以及单任务学习 [92, 40]。相比之下, 从具备丰富表征知识的大型机器人 foundation model 出发进行微调 (fine-tuning), 有望显著缩小搜索空间, 并使模型学会更复杂的运动模式, 从而让在更通用任务和环境上的训练成为可能。

本文通过一项系统性研究来探讨这一问题。为了高效实现可扩展的 RL 训练, 我们提出 VLA-RL: 一个利用在线 RL 提升预训练自回归 VLA 的统一框架。具体而言, 在统一视角下, 我们首先给出一种适用于自回归 VLA 训练的通用 RL 形式化, 将通用机器人操纵轨迹建模为多模态、多轮对话。针对机器人大动作空间中稀疏奖励带来的挑战, 我们进一步实例化了一个机器人过程 reward model: 它本质上是一个在自动抽取的伪奖励标签上进行微调的 vision-language model。基于 VLA-RL, 我们还总结出一系列系统性的实现改进, 包括任务选择、GPU 负载均衡环境、batch decoding, 以及 critic warmup, 以提升训练的稳定性与效率。实验上, 我们采用 OpenVLA-7B [39] 作为基础 VLA, 并将所提出的方法应用于 LIBERO [45] 中的 40 个高难度机器人任务。结果表明, VLA-RL 相比基础模型带来了 4.5% 的显著提升, 甚至能够匹配 π -FAST [57] 等先进商业模型的性能。进一步地, 随着测试时计算量增加, VLA-RL 的性能还能持续提升, 这为机器人领域中 inference scaling laws [64] 的存在提供了初步证据。

3 2 Related Work

3.1 机器人基础模型

机器人基础模型通过在大规模多任务、多 embodiment 机器人数据集 [15, 20, 16, 18, 72, 37] 上进行多任务训练 [9, 8, 68, 39, 44, 35, 36, 19, 6, 7, 57, 32, 48, 46], 在学习可泛化的机器人行为方面展现出显著潜力。尤其是 OpenVLA-7B [39], 它在高容量 vision-language model 上进行微调 (fine-tuning), 将机器人动作表示为模型词表中的 language token 来生成, 并在多种任务上表现出令人印象深刻的泛化能力。然而, 这类通过模仿学习训练得到的 VLA 在部署时仍面临明显挑战: 一旦遇到离线专家 demonstration 未覆盖的 OOD 场景, 性能往往会显著下降。

3.2 面向机器人模型的强化学习

尽管强化学习已经在机器人领域取得了大量成果 [66], 但从零开始实现 RL 方法通常需要精心设计 reward 和训练范式 [2, 29, 85, 91, 28, 89, 22]。因此, 研究者系统地探索了如何利用预训练模型来提升 RL 性能 [65, 38, 67, 54, 25, 24, 34, 49, 4, 93, 76, 70, 27, 77]。不过, 已有工作大多要求在整个微调过程中始终可以访问完整的离线数据集 [54, 1, 25, 24, 34, 71, 40, 49, 60, 5]。

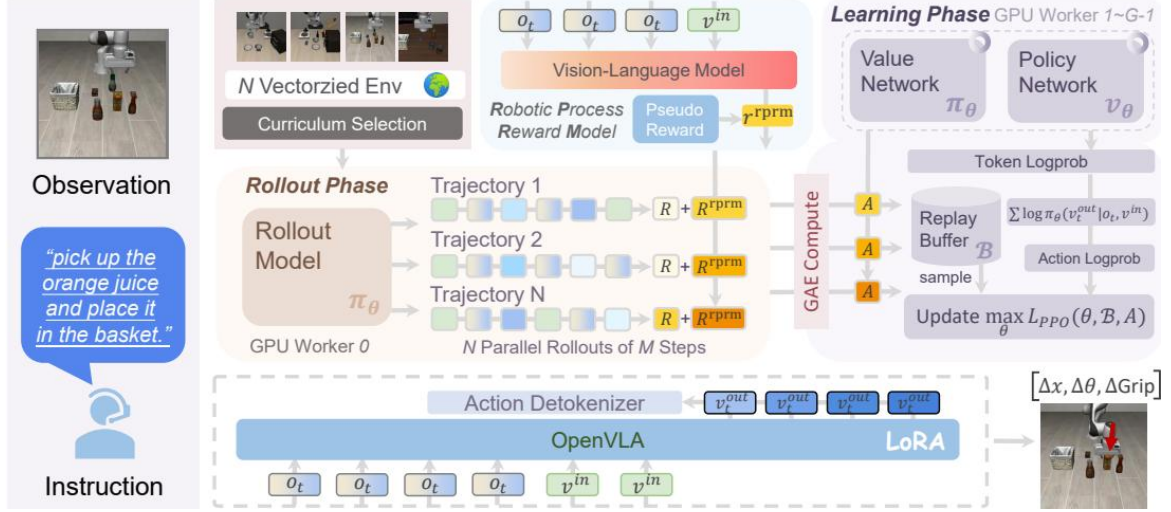


图 2: VLA-RL 的整体流程。该方法由一个基于 transformer 的 policy、一个同构的 value model、一个冻结的 robotic process reward model, 以及向量化环境组成。

此外, 其他一些已有方法仅在具有简化状态表征的环境中进行测试 [60, 24, 54], 使用较为朴素的神经网络结构 [60, 70, 1, 50, 51], 并局限于范围狭窄的单任务训练范式 [92, 40]。相比之下, VLA-RL 探索了基于大规模机器人基础模型进行轨迹级 RL 微调的方案。借助这些模型丰富的表征知识, 方法能够显著缩小搜索空间, 并帮助模型学习复杂运动模式, 从而使其能够在更通用的任务与环境上开展训练。

3.3 面向大模型的强化学习

强化学习已经通过一系列方法显著提升了 LLM 的推理能力, 而这些方法也直接启发了我们的工作 [62, 17, 88, 43, 56, 11, 75, 82, 47, 31, 3]。这一研究脉络可追溯至 STaR [84], 并进一步发展出 reinforced self-training [21] 和 rejection fine-tuning [83] 等方法; 它们依赖 LLM 自身生成的解答, 实现交互式自我更新。为了支持更自主的探索与利用, PPO [62] 等 policy optimization 方法已成为当前实践中的主流选择。例如, GRPO [17] 强化了系统性问题求解能力, 这一点对序列决策尤其相关。能够增强推理能力的方法还包括 Process Reward Model [88, 43], 它们用于评估中间推理过程; 以及基于对话的训练方法 [56, 11, 75], 它们针对多轮交互进行优化。近期提出的 RL 技术 [82, 47, 31] 进一步表明, 不同的实现细节会显著影响语言模型推理能力的扩展效果。我们的工作将这些进展引入机器人领域: 通过将轨迹级优化表述为一个多模态、多轮对话过程, 使机器人能够受益于那些已在大语言模型中被证明有效的 RL 框架。

4 VLA-RL

本节首先简要介绍问题定义以及开源 VLA 模型的相关预备知识（第 3.1 节）。随后，我们给出整体 pipeline 的概览（第 3.2 节）。接着，我们将一般机器人操纵任务形式化为一种多模态、多轮对话过程（第 3.3 节）。在此基础上，我们进一步提出用于奖励稠密化的机器人过程奖励模型（robotic process reward model, 第 3.4 节）。最后，在实际实现层面，我们构建并介绍 VLA-RL 系统（第 3.5 节），并总结若干关键发现，这些发现有助于通过 RL 实现可扩展的 VLA 训练。

4.1 3.1 预备知识

通用机器人操纵一直是机器人领域长期关注的核心问题。智能体需要根据视觉观测以及指明当前任务的人类指令，交互式地决定下一步机器人动作（即 end-effector 的位姿），以完成多样化任务。近年来，大容量预训练视觉-语言模型在大范围语言条件操纵任务上展现出较强的泛化能力。其中，OpenVLA-7B [39] 是具有代表性的开源 VLA，因此被用作我们方法的基础模型。其核心是一个自回归 LLM——Llama-2-7B [69]，并配有一个双流视觉 encoder，由预训练的 SigLIP [86] 和 DinoV2 [55] 组成。在每个时间步 t ，模型接收第三人称相机采集的图像 $\mathbf{o}_t$ 和人类指令 $\mathbf{v}_t^{\text{in}}$ 作为输入，输出动作 token 序列 $\hat{\mathbf{v}}_t^{\text{out}} \in \mathcal{V}^n$ 。其中，每个动作 token 表示机器人动作空间某一维上的一个离散 bin。最终，利用后处理函数 f 从该序列中提取出机器人动作，得到 $\mathbf{a}_t = f(\hat{\mathbf{v}}_t^{\text{out}})$ 。然而，优化这类自回归 VLA 在算法和系统两个层面都面临挑战，例如面向通用操纵的 RL、稀疏奖励问题，以及大规模评测与优化等。

算法 1: VLA-RL 输入：环境数量 N ，参数为 θ_0 的初始 VLA policy，参数为 ϕ 的已训练机器人过程奖励模型，后处理函数 f ，每次更新的步数 $\hat{M}$ ，折扣因子 γ ，GAE 参数 λ_{GAE} 。

输出：改进后的 VLA policy θ_K 。

1. $\text{envs} \leftarrow \text{VecEnv}(\text{num_envs} = N)$
2. $\mathbf{o} \leftarrow \text{envs.reset}()$
3. $\mathbf{d}_0 \leftarrow \{0\}^N$ 终止状态指示符
4. **for** $k = 0$ **to** $K - 1$ **do**
5. $B_k \leftarrow \emptyset$
6. # Rollout 阶段：从 N 个并行环境中采集轨迹
7. **for** $t = 0$ **to** $M - 1$ **do**
8. $\mathbf{v}_t^{\text{in}} \leftarrow h(\mathbf{o}_t)$ 对所有观测应用视觉-文本处理器
9. $\mathbf{v}_t^{\text{out}} \sim \pi_{\theta_k}(\mathbf{o}_t, \mathbf{v}_t^{\text{in}})$ policy 模型生成动作 token
10. $\mathbf{a}_t \leftarrow f(\mathbf{v}_t^{\text{out}})$ 提取动作
11. # 将动作对数概率计算为各 token 对数概率之和
12. $\log \pi_{\theta_k}(\mathbf{a}_t | \mathbf{o}_t, \mathbf{v}_t^{\text{in}}) = \sum_i \log \pi_{\theta_k}(\mathbf{v}_{t,i}^{\text{out}} | \mathbf{o}_t, \mathbf{v}_t^{\text{in}})$
13. $\mathbf{V}_t \leftarrow V_{\theta_k}(\mathbf{o}_t, \mathbf{v}_t^{\text{in}})$ value 模型进行估计

14. $\mathbf{o}_{t+1}, \mathbf{r}_t^{\text{sparse}}, \mathbf{d}_{t+1}, \mathbf{i}_t \leftarrow \text{envs.step}(\mathbf{a}_t)$
15. $\mathbf{r}_t^{\text{prm}} \leftarrow R_\phi(\mathbf{o}_t, \mathbf{a}_t)$ 机器人过程奖励模型
16. $\mathbf{r}_t \leftarrow \mathbf{r}_t^{\text{sparse}} + \mathbf{r}_t^{\text{prm}}$
17. $B_k \leftarrow B_k \cup \{(\mathbf{o}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{d}_t, \mathbf{v}_t^{\text{out}}, \log \pi_{\theta_k}(\mathbf{a}_t | \mathbf{o}_t, \mathbf{v}_t^{\text{in}}), \mathbf{V}_t)\}$
18. $\mathbf{A}^{\text{GAE}} \leftarrow \text{GAE}(B_k, \mathbf{V}_M, \gamma, \lambda_{\text{GAE}})$ 计算优势
19. $\theta_{k+1} \leftarrow \text{PPO}(\theta_k, B_k, \mathbf{A}^{\text{GAE}})$ 学习阶段：更新 policy 参数
20. **return** θ_K

4.2 3.2 Overall Pipeline

VLA-RL 的整体流程如图 2 所示。我们提出了一个兼具算法设计与系统实现的完整框架，用于通过 RL 训练自回归 VLA。整个系统由三个模型组成：需要在常用 actor-critic 框架下训练的 policy model 和 value model，以及一个冻结的 robotic process reward model，用于将环境提供的稀疏奖励稠密化。

在算法层面，我们将自回归 VLA-RL 的训练过程表述为一个多模态、多轮对话问题。在系统层面，GPU-balanced vectorized environment、batch decoding、curriculum selection strategy 以及 critic warmup 等技术进一步提升了系统的训练效率与稳定性。最终，经过后训练的 VLA model 能够通过优化期望奖励生成可行的动作，从而成功完成多种操纵任务。

4.3 3.3 将通用机器人操纵建模为多轮对话

为了将 RL 扩展到面向通用操纵的自回归 VLA 优化中，我们首先将马尔可夫决策过程表述为多轮对话。设 V 表示离散且有限的词表 token 集合。 $\mathcal{V}^m$ 和 $\mathcal{V}^n$ 分别表示可能的输入文本序列空间与输出文本序列空间，其中 m 和 n 分别给出输入与输出序列的最大长度。我们将状态空间定义为笛卡尔积 $S = \mathcal{O} \times \mathcal{V}^m$ ，其中 $\mathcal{O}$ 为图像空间。动作空间则定义为由 VLA 生成的全部可能输出话语集合，即 $\mathcal{V}^n$ 。据此，参数为 θ 的 VLA policy 可形式化为映射 $\pi_\theta : \mathcal{O} \times \mathcal{V}^m \rightarrow \mathcal{V}^n$ 。在每个时间步，给定输入图像 o_t 与 prompt $\mathbf{v}_t^{\text{in}}$ ，该 policy 为输出序列 $\mathbf{v}_t^{\text{out}}$ 赋予概率 $\pi_\theta(\mathbf{v}_t^{\text{out}} | o_t, \mathbf{v}_t^{\text{in}}) \in [0, 1]$ 。环境转移由函数 $\mathcal{T} : S \times A \rightarrow S$ 决定，用于描述每次动作执行后状态如何演化。环境奖励函数 $R : S \rightarrow \mathbb{R}$ 则以每次动作后的 r_t 来衡量结果质量。在长度为 T 的轨迹上，优化目标是最大化折扣奖励和，[formula]，其中 γ 为折扣因子。我们采用 Proximal Policy Optimization (PPO) [62] 来实现稳定的 policy 优化。

Rollout 阶段 首先，我们将更新后的 LoRA [26] 权重与原始 checkpoint 合并，并广播到推理引擎。随后，agent 按照当前 policy $\pi_{\theta_{\text{old}}}$ 与环境交互，生成由状态、动作和奖励组成的序列（即轨迹）。对于自回归模型，动作序列的对数概率可以分解为 token 级对数概率之和：

$$\log \pi_\theta(\mathbf{a}_t | \mathbf{o}_t, \mathbf{v}_t^{\text{in}}) = \sum_{i=1}^{|\mathcal{A}|} \log \pi_\theta(\mathbf{v}_{t,i}^{\text{out}} | \mathbf{o}_t, \mathbf{v}_t^{\text{in}}) \quad (1)$$

其中， $|\mathcal{A}| = 7$ 表示 OpenVLA 动作空间的自由度 (DoF) 数量。

学习阶段 PPO 的目标函数采用带裁剪的 importance sampling, 以保证更新过程的稳定性:

$$\mathcal{L}_{\text{ppo}}(\theta) = \mathbb{E}_t \left[\min \left(\frac{\pi_{\theta}(\mathbf{a}_t | \mathbf{o}_t, \mathbf{v}_t^{\text{in}})}{\pi_{\theta_{\text{old}}}(\mathbf{a}_t | \mathbf{o}_t, \mathbf{v}_t^{\text{in}})} A_t, \text{clip} \left(\frac{\pi_{\theta}(\mathbf{a}_t | \mathbf{o}_t, \mathbf{v}_t^{\text{in}})}{\pi_{\theta_{\text{old}}}(\mathbf{a}_t | \mathbf{o}_t, \mathbf{v}_t^{\text{in}})}, 1 - \epsilon, 1 + \epsilon \right) A_t \right) \right] \quad (2)$$

其中, ϵ 为裁剪参数, 用于限制新旧 policy 概率比值的变化范围, 从而避免 policy 更新过大。对于每个状态, 其优势值 A_t 通过 Generalized Advantage Estimation (GAE) [61] 计算。整体流程总结于算法 1。

4.4 3.4 Robotic Process Reward Model

奖励建模是将强化学习 (RL) 应用于通用操纵的关键, 其需要满足两点: (1) 在天然反馈稀疏的环境中提供稠密奖励; (2) 避免 reward hacking, 即智能体以非预期方式利用奖励函数。为此, 我们提出 Robotic Process Reward Model, 一种新的奖励稠密化方法, 并使其与 VLA 的 token 生成过程保持一致。

将奖励建模重写为 next-token prediction。 传统机器人强化学习通常受限于奖励稀疏问题: 奖励往往是仅在任务完成时给出的二值信号。我们将奖励建模重写为 next-token prediction 问题, 从而利用预训练视觉语言模型的自回归特性。给定由状态与动作构成的轨迹, Robotic Process Reward Model (RPRM) 预测成功动作序列的似然。其训练目标是最大化潜在优质动作 token 的对数似然, 并使用一个指示任务完成进度的伪奖励信号进行加权:

$$\mathcal{L}_{\text{rprm}}(\phi) = -\mathbb{E}_t \left[\sum_{j=1} \log p_{\phi}(\mathbf{v}_{t,j}^{\text{rprm}} | \mathbf{v}_{t,<j}^{\text{out}}, \mathbf{o}_t, \mathbf{v}_t^{\text{in}}) \right] \quad (3)$$

其中, ϕ 表示 robotic process reward model 的参数, $p_{\phi}(\mathbf{v}_j^{\text{rprm}} | \cdot)$ 表示 robotic process reward model 对下一个 token $\mathbf{v}_j^{\text{rprm}}$ 的预测概率。

自主伪奖励标签生成。 为了在不过度依赖人工标注的情况下有效训练 robotic process reward model, 我们设计了一条自主标签生成流程, 从成功轨迹中构造高质量的伪奖励标签。具体包括:

1. **里程碑分段 (Milestone Segmentation):** 我们从专家 demonstration 与先前模型运行结果中收集多样化的成功轨迹数据集。随后, 根据 gripper 开合程度的显著变化将轨迹切分为多个子任务, 因为这类变化通常对应某个功能性步骤的完成。
2. **进度标注 (Progress Labeling):** 在每个分段后的子任务内部, 我们进一步识别 end-effector 速度接近于零的关键帧。这些时刻通常对应于稳定状态, 或某些细粒度动作的完成。对于导向这些关键帧的 VLA 动作序列, 我们赋予正的伪奖励。

最终奖励由真实的稀疏奖励与 robotic process reward model 预测的奖励直接相加得到。我们的实证分析表明, 该方法在与真实任务成功保持较强相关性的同时, 能够显著加速学习过程。

4.5 3.5 VLA-RL 系统

由于 RL 的性能高度依赖具体实现细节, 这里分享我们在本项目中采用的一些技巧, 用于提升学习效率与训练稳定性。

课程选择策略 (Curriculum Selection Strategy) 我们实现了一种自适应课程机制，根据 agent 当前的能力动态选择任务。每个任务由一条 instruction 和一个初始状态组成。对于每个任务，我们维护其成功率 s_j ，并按如下方式计算采样概率：

$$P(task_j) \propto \exp((0.5 - s_j)/\tau) \quad (4)$$

其中， τ 用于控制探索强度。该公式会优先采样成功率接近 50% 的任务，因为这类任务正处于 agent 能力边界的前沿；同时，它也保留了对已掌握任务和高难任务的采样机会，从而提升样本效率与泛化能力。

Critic Warmup 当从头开始训练 value model (critic) 时，模型在初期往往会给出不准确的价值估计，这可能在训练早期误导 policy 更新。为缓解这一问题，我们引入了 critic warmup 阶段：先使用经过模仿学习预训练的 policy 收集初始轨迹，仅训练 value network 若干轮，随后再开始 policy 与 value 的联合优化。

GPU 均衡的向量化环境 (GPU-balanced Vectorized Environments) 我们实现了多个向量化环境以支持并行 rollout，其中每张训练 GPU 持有一部分环境实例。现代渲染器通常依赖 GPU 加速，但随着向量化环境数量增加，GPU 显存占用也会显著增长。为此，我们让每个 GPU worker 维护并交互于其独立的一组环境，并基于这些环境进行学习。同时，我们通过 all_reduce 操作汇聚所有 worker 的环境状态，供推理引擎使用。

基础设施 (Infrastructure) PPO 训练基础设施采用 bfloat16，以便将模型装入显存。在总计 G 张 GPU 的配置下，我们专门划出 1 张 GPU，借助 vLLM [41] 加速进行 inference；其余 $G - 1$ 张 GPU 则通过 Ray [53] 用于学习，这一做法与 OpenRLHF [30] 和 open-instruct [42] 类似。在我们的代码库中，我们已在 vLLM 插件中实现 OpenVLA [39]，从而避免使用原始 Huggingface transformers 的 generation 函数，因为该函数在大 batch size 场景下会产生错误结果。分布式训练过程由 PyTorch Fully Sharded Data Parallel (FSDP) [90] 管理，以支持大规模训练。

5 4 实验

在实验部分，我们主要关注以下问题：

1. 在常用的机器人操纵 benchmark 上，VLA-RL 的表现如何？
2. 在通用机器人操纵场景中，VLA-RL 是否能够随着测试时计算量的增加而扩展？
3. 从训练数据的状态覆盖与动作覆盖角度看，为什么 VLA-RL 相比 behavior cloning 具有更强的鲁棒性？
4. 所提出的技术设计与具体实现细节会如何影响 VLA-RL 的性能？

接下来的各节将通过严格控制变量的实验，对上述关键问题进行详细分析。

5.1 4.1 主要实验

实验设置 本节考察如何将 VLA-RL 适配到仿真机器人配置与任务中，具体采用 LIBERO 基准 [45]。LIBERO 基准 [45] 包含四个具有挑战性的任务套件：LIBERO-Spatial、LIBERO-Object、LIBERO-Goal

和 LIBERO-Long (或 LIBERO-10), 分别关注不同的空间关系、物体类别、目标设定以及更长时序的序列挑战。在实验中, 我们以 OpenVLA-7B [39] 为基础模型, 并针对每个任务套件从经监督微调 (SFT) 训练得到的模型出发开展 RL。测试阶段中, 所有对比方法都在每个任务套件上评估 500 个 episode。图 3 展示了该基准中的若干成功样例。

基线方法 在 RL 训练中, 我们采用 [39] 公开发布的 checkpoint 作为基础 SFT 模型。除 SFT baseline 外, 我们还报告了从头训练的 Diffusion Policy [12]、经过微调的基于 diffusion 的 VLA 模型 Octo [68], 以及使用 Direct Preference Optimization (DPO) [59] 训练的 GRAPE [89] 的性能, 以便进行更充分的参考比较。在评价指标方面, 我们按照 [39] 的设置, 报告平均成功率 (success rate, SR) 和平均排名。

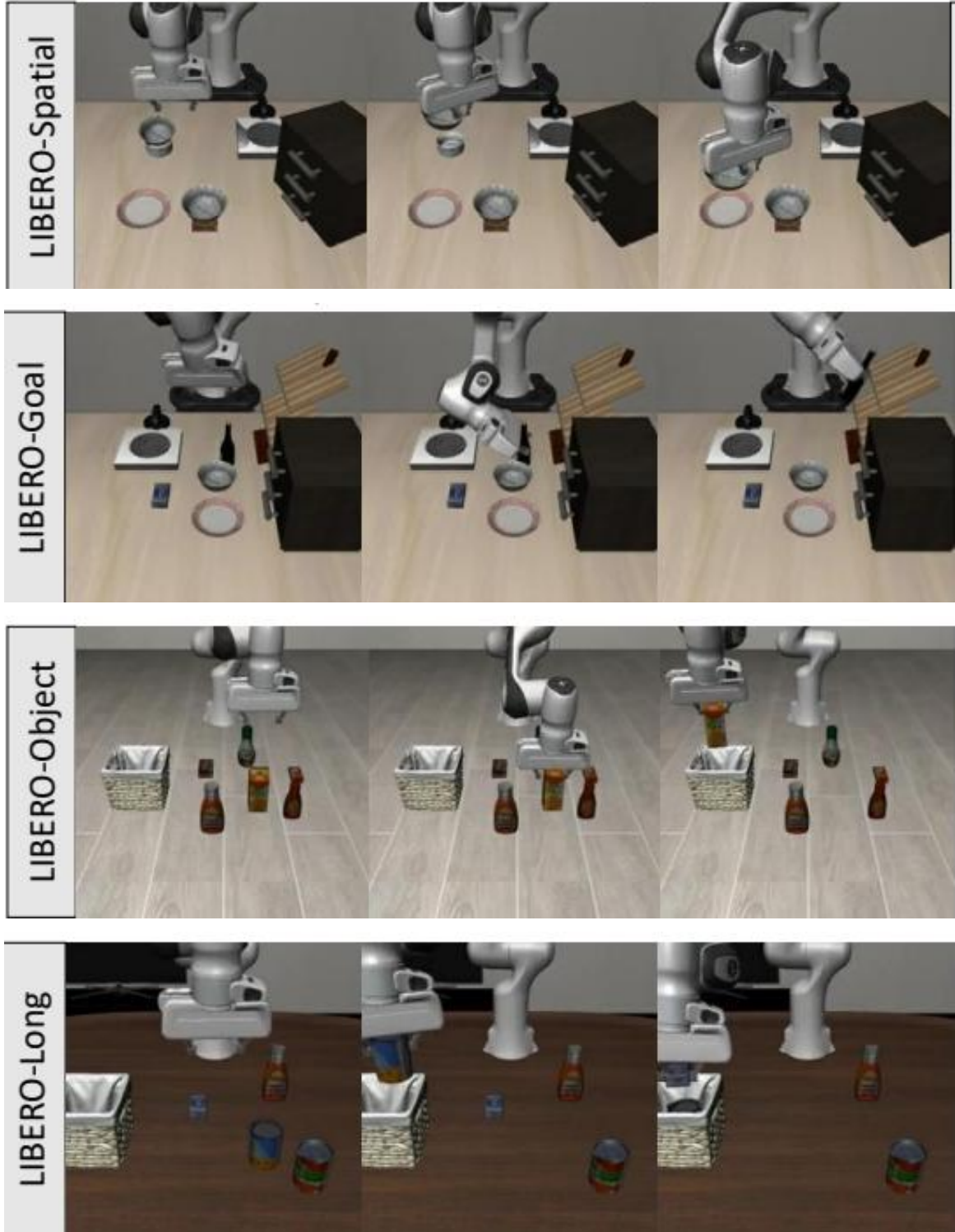
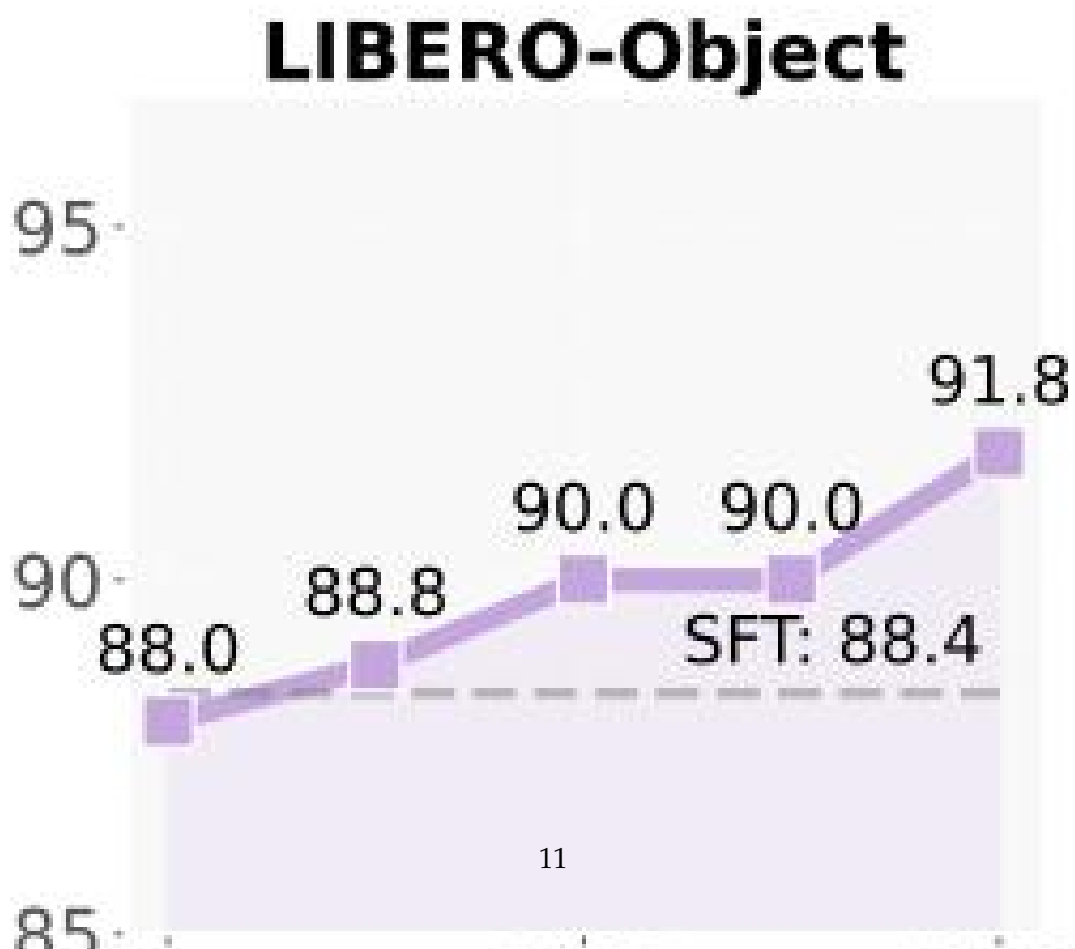
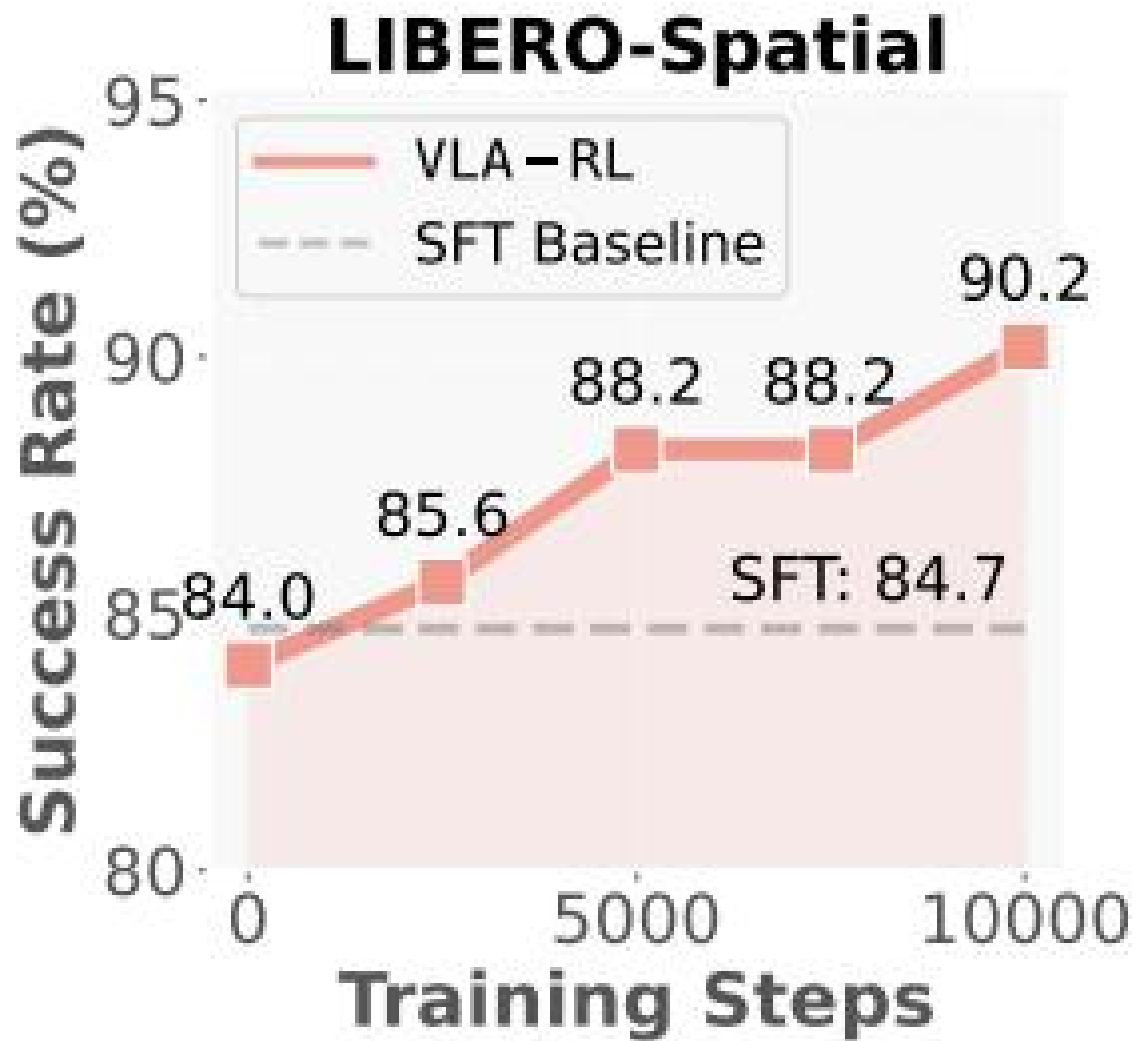


图 3: 环境与任务示例。对于仿真实验，我们在常用的机器人操纵基准 LIBERO 上进行评估。该基准包含四个任务套件，分别聚焦于不同类型的挑战。图中示例指令包括：“拿起放在小碟上的黑色碗并将其放到盘子上”、“拿起橙汁并将其放入篮子中”、“将酒瓶放到架子上”，以及“将字母汤和奶油奶酪盒都放入篮子中”。

性能对比 表 1 给出了 RL 训练完成后在 LIBERO 上的实验结果。相较于采用 SFT 和 DPO 的 OpenVLA-7B checkpoint, VLA-RL 分别带来了 4.5% 和 1.8% 的显著提升, 表明在线 RL 以及本文提出的 VLA-RL 框架是有效的。值得注意的是, 仅经过 48 个 GPU 小时的 RL 训练后, 微调后的 OpenVLA-7B 就达到了先进商业模型 π_0 -FAST 的性能水平; 而后者依赖高质量 SFT 数据进行训练。同时, 性能曲线仍

保持持续上升趋势。上述结果表明，通过大规模 RL 增强高容量 VLA 具有近乎无限的潜力。



测试时 scaling 图 4 给出了四个 LIBERO 任务在 RL 训练过程中的测试成功率变化。随着测试时优化的推进，四个任务套件上的评估成功率均持续提升，这表明机器人领域中的 inference scaling laws [64] 已开始显现出早期迹象。

5.2 4.2 训练动态

在大模型上开展强化学习时，需要仔细监控关键指标，以便发现系统中的偏差并据此进行改进 [82]。遵循常见做法 [82, 42]，我们报告并分析了学习过程中的训练动态，以分享关于使用 RL 训练 VLA 的若干关键发现。结果如图 5 所示。

生成 episode 的长度 我们观察到，成功的 VLA-RL 训练会使 episode 长度逐步缩短，这表明模型学会了操纵任务更高效的动作序列。这一点与基于 LLM 的 RL 不同；在后者中，更长的序列往往与更强的推理能力相关。

训练过程中的 reward 动态 reward 的变化趋势表明，训练期间性能持续提升，并会周期性出现平台期；这些平台期对应于课程学习中不同任务难度之间的切换。reward 的提升与物理任务的成功率 (success rate) 高度相关，这说明我们的机器人过程奖励模型能够有效刻画操纵能力提升中的真实进展。

表 1: LIBERO 仿真基准结果。我们统计了各方法在 LIBERO 四个任务套件上的平均成功率 (success rate) 与排名，评估基于每种方法 500 个 episode。每个任务套件内的方法排名范围为 1-5，其中 1 表示性能最佳，5 表示最差。我们还加入了先进的商业自回归模型 π 0-FAST [57] 作为参考。

方法	LIBERO-Spatial		LIBERO-Object		LIBERO-Goal		LIBERO-Long		平均排名
	SR ($\uparrow$)	Rank ($\downarrow$)	SR ($\uparrow$)	Rank ($\downarrow$)	SR ($\uparrow$)	Rank ($\downarrow$)	SR ($\uparrow$)	Rank ($\downarrow$)	
Diffusion Policy [12]	78.3%	5	92.5%	1	68.3%	5	50.5%	5	4.0
Octo (SFT) [68]	78.9%	4	85.7%	5	84.6%	1	51.1%	4	3.5
OpenVLA (SFT) [39]	84.7%	3	88.4%	4	79.2%	4	53.7%	3	3.5
GRAPE (DPO) [89]	87.6%	2	91.2%	3	82.2%	2	55.8%	2	2.3
π 0-FAST [57]	96.4%	–	96.8%	–	88.6%	–	60.2%	–	–
VLA-RL (Ours)	90.2%	1	91.8%	2	82.2%	2	59.8%	1	1.5

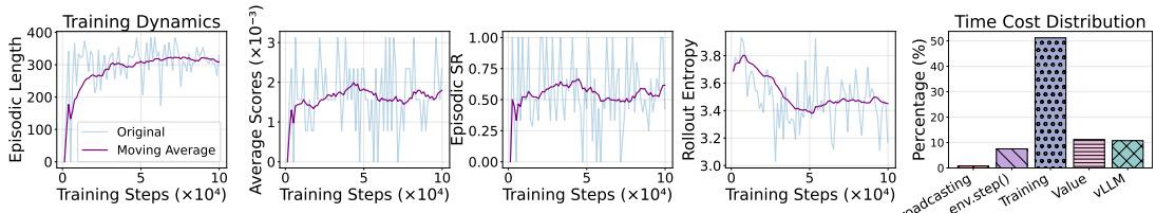


图 5: 训练动态。我们在 LIBERO-Long 上绘制了训练过程中生成 episode 的长度、reward 动态以及 rollout entropy。

policy 的 rollout entropy 合适的动作 entropy 至关重要：entropy 过低会限制探索，而 entropy 过高则会导致改进过程不稳定。我们的实现将 entropy 维持在适中的水平，并随着训练推进逐渐降低，同时保留了训练初期所需的探索能力。

时间分析 图 5 右侧子图展示了系统各组成部分的时间开销分布。借助 GPU 负载均衡的向量化环境以及 vLLM 加速, 环境演化和模型 rollout 所消耗的时间已显著降低。因此, 当前主要瓶颈转移到了训练阶段, 这表明若能进一步提升训练效率, 整体性能仍有望获得显著收益。

5.3 4.3 消融实验

我们进一步在 LIBERO-Spatial 任务套件上开展了额外的消融实验, 以更全面地验证 VLA-RL 中所提出设计选择的有效性, 涵盖技术层面与实现细节两个方面。对比实验结果如表 2 所示。

表 2: 消融实验。我们报告了 LIBERO-Spatial 上最终的平均成功率 (success rate)。从 VLA-RL 中去除任意一个稳定化技术, 都会导致性能快速退化, 这表明 VLA-RL 中引入的每项技术都至关重要。

超参数 / 设置	LIBERO-Spatial
VLA-RL	90.2
去除 Robotics PRM	85.8
去除 Curriculum	88.0
Temperature = 1.5 $\rightarrow$ 1.0	85.8
Critic Warmup Steps = 5 $\rightarrow$ 0	80.0
LR = $2e^{-5}$ $\rightarrow$ $2e^{-4}$	0.2

奖励稠密化 (reward densification) 的选择。 相较于稀疏奖励 baseline, 本文提出的 robotic process reward model 将成功率从 85.8% 提升到 90.2%, 说明奖励稠密化具有重要意义。robotic process reward model 能提供更频繁、信息量更高的反馈信号, 从而引导智能体更有效地完成任务; 这一点在长时程任务中尤为关键, 因为稀疏奖励往往会导致探索效率低下。

Curriculum 选择策略。 基于 Curriculum 的任务选择优于均匀随机选择, 将成功率从 88.0% 提升到 90.2%。通过根据智能体表现逐步提升任务复杂度, policy 可以先在较简单的任务上更有效地学习, 再处理更困难的任务。这种方式能够减轻灾难性遗忘, 并提升在整体任务分布上的泛化能力。

采样温度 (sampling temperature) 的选择。 较低的温度会抑制探索, 使训练更容易收敛到次优 policy。为验证这一点, 我们将温度从 1.5 降至 1.0, 成功率由 90.2% 降低到 85.8%, 说明随机性的降低会损害 policy 的评估与优化过程。

Critic warmup 的选择。 在更新 policy 之前, 先对 value model 进行 warmup, 可将成功率从 80.0% 提升到 90.2%。这种预训练能够为 policy gradient 提供更准确的反馈; 相反, 若不进行 warmup, 初期噪声较大的 value 估计会对 policy 学习产生负面影响。

学习率 (learning rate) 的选择。 较低的学习率 ($2e^{-5}$) 能达到较好的性能。更高的学习率 ($2e^{-4}$) 会引发训练不稳定, 这与文献 [28] 中的观察一致。不过, 较低的学习率也会减慢收敛速度, 这进一步说明了 RL 系统中实现细节的重要性。

总体来看，消融实验凸显了 VLA-RL 中各个组成部分的必要性。完整方法取得了最高的 90.2% 成功率；去除任意单一组件都会带来明显的性能下降。这表明，本文提出的各项技术并非彼此独立，而是以协同方式共同提升了整体方法的有效性。

5.4 4.4 RL vs. SFT

动作覆盖范围分析。 我们在图 6 中可视化了所收集动作在投影到 XY 平面后的分布。

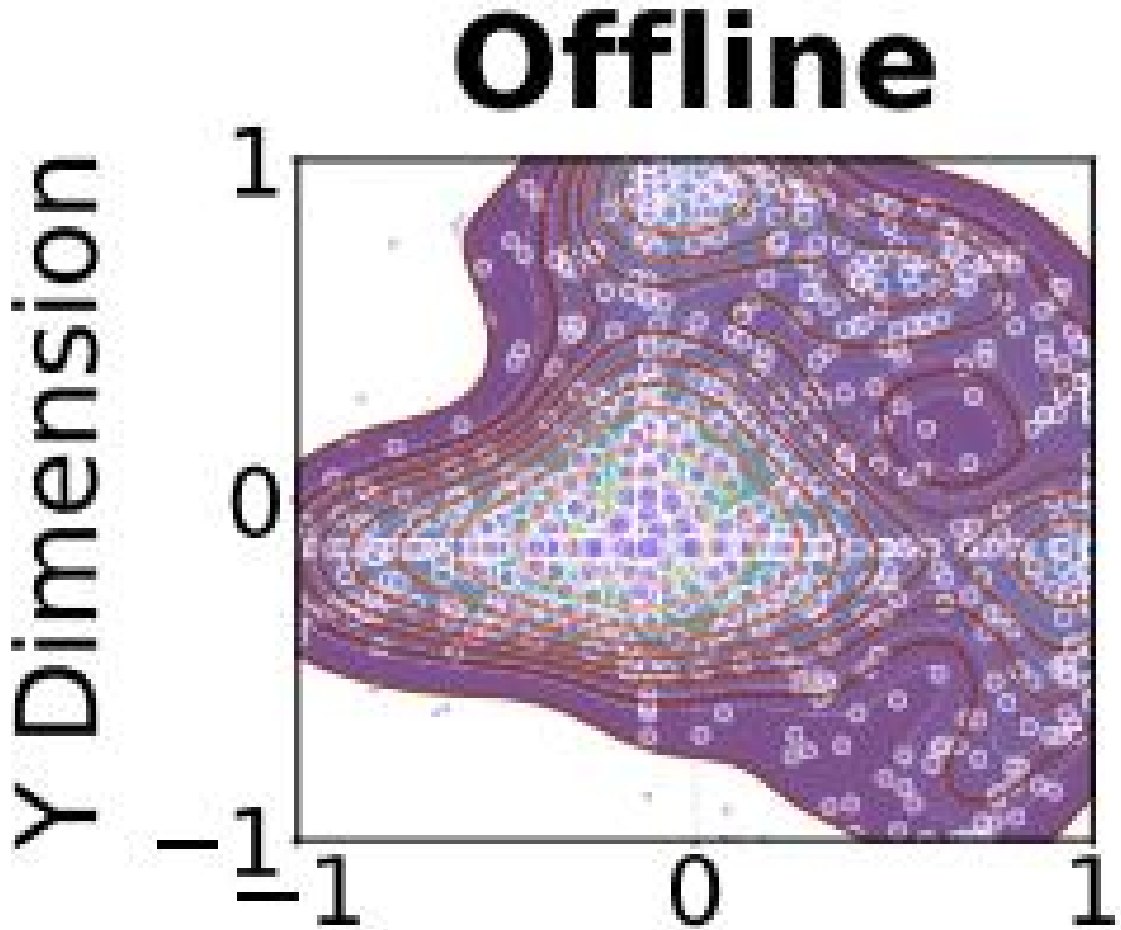


图 6: SFT 与 RL 方法的动作覆盖范围。我们在 LIBERO-Spatial 上可视化了所收集动作（相对 end-effector 位姿）的前两个维度。

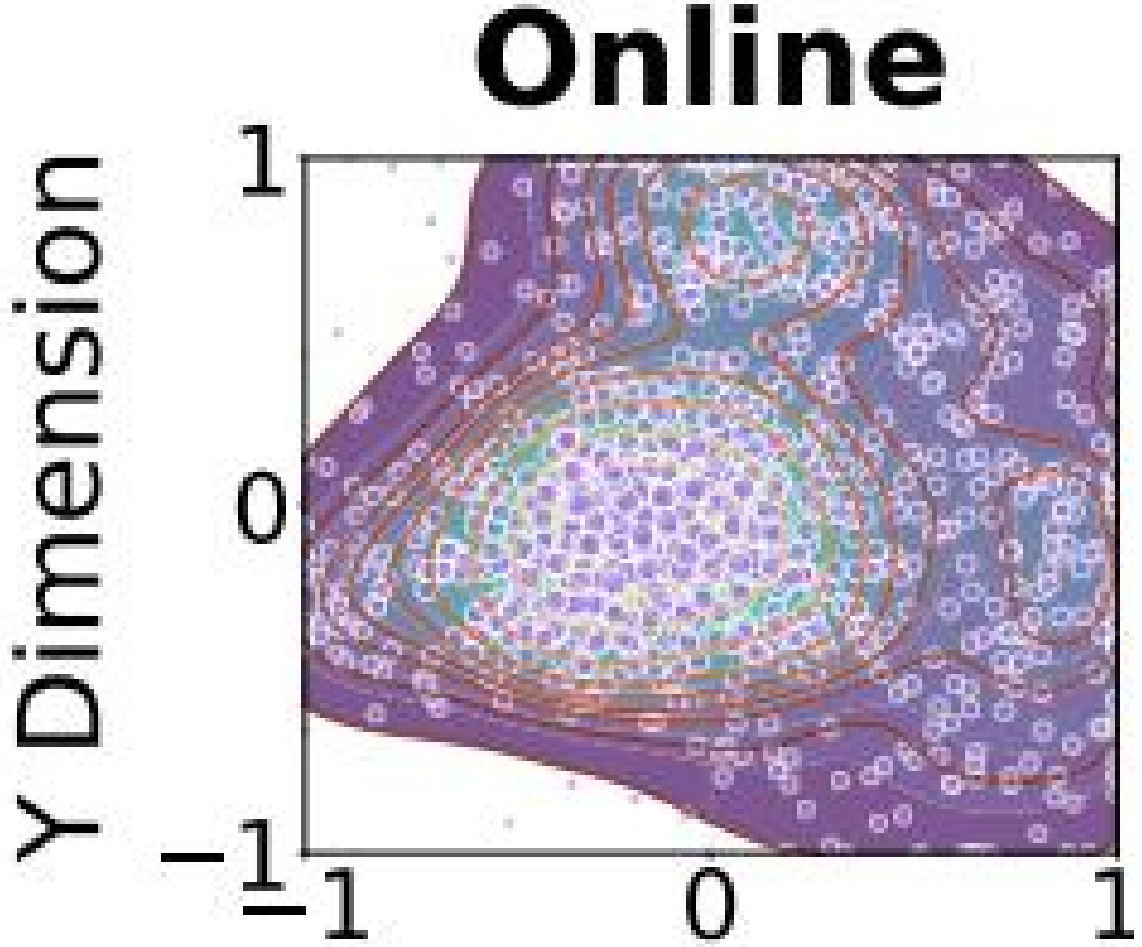


图 7: SFT 与 RL 方法的动作覆盖范围。我们在 LIBERO-Spatial 上可视化了所收集动作（相对 end-effector 位姿）的前两个维度。

在 LIBERO-Spatial 中，机械臂需要精确运动到指定接触点。对动作分布的分析表明，离线与在线数据采集在分布形态上存在显著差异。expert 动作通常集中在动作空间中心附近，体现出对若干常见位置的偏好，并且往往呈现重复的运动模式。相比之下，RL agent 生成的动作在整个动作空间中分布得更均匀，这使 RL policy 能够适应更广泛的状态。因此，基于 RL 采集数据训练得到的 policy，比基于离线数据训练的 SFT 模型表现出更强的鲁棒性。

案例分析。 为探究强化学习在哪些场景下表现得尤为有效，我们分析了 LIBERO-Goal 中 RL 与 SFT 模型的 rollout 轨迹。如图 7 所示，给定的指令为：“pick up the black bowl on the wooden cabinet and place it on the plate”，该任务需要细粒度交互。采用 VLA-RL 微调（fine-tuning）后的模型能够成功抓取瓶子，而 SFT baseline 由于抓取点发生偏移而失败。由此可见，使用 RL 采集数据训练的 policy 在 contact-rich 任务中能够更稳定地缓解对齐误差，并减少抓取过程中 gripper 过早闭合的问题，这一现象也与文献 [78] 中的观察一致。

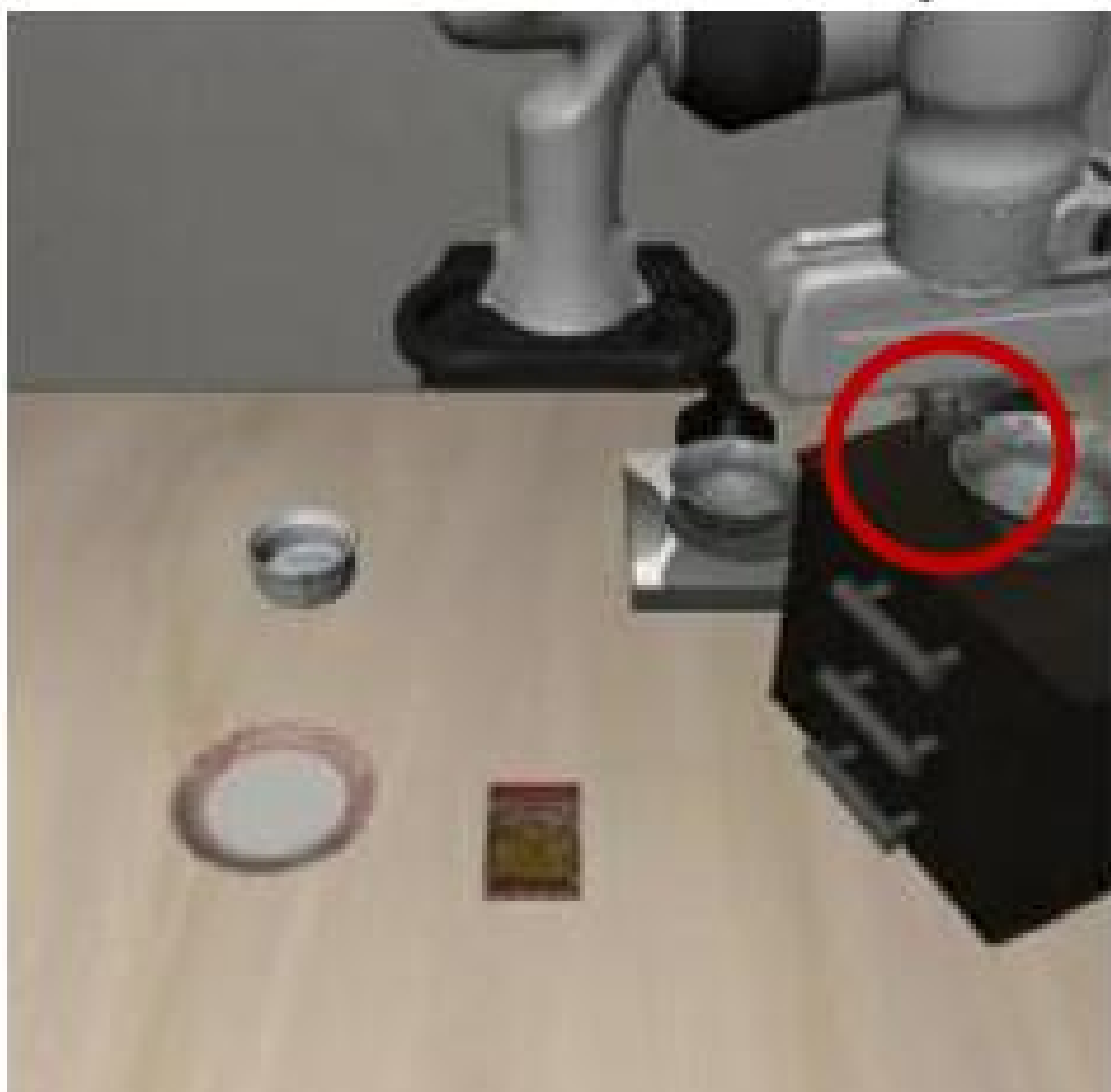


图 8: 案例分析。我们展示了初始 SFT baseline 以及 VLA-RL (本文方法) 生成的机器人轨迹。



图 9: 案例分析。我们展示了初始 SFT baseline 以及 VLA-RL（本文方法）生成的机器人轨迹。

6 5 Conclusions and Limitations

结论。本文提出 VLA-RL，这是一种可扩展的强化学习框架，旨在通过在线 policy 优化增强预训练 VLA 的能力。我们将通用机器人操纵建模为多模态多轮对话，从而使 VLA 模型能够突破离线 demonstration 数据的局限，进行更充分的探索。针对复杂任务中的稀疏奖励问题，我们基于微调 (fine-tuning) 的 vision-language model 构建了统一的机器人过程奖励模型，并验证了其有效性。结合一系列系统性的设计选择，本文方法取得了显著的性能提升：在 LIBERO benchmark 上，相比强有力的模仿学习 baseline OpenVLA-7B，性能提高了 4.5%。更重要的是，我们观察到模型性能会随着测试时优化而持续提升，这表明其中可能存在一种新兴规律，类似于语言模型中的 inference scaling laws。

局限性。尽管结果令人鼓舞，我们的方法仍面临若干挑战。例如，文中提出的用于提取伪奖励标签的启发式策略，可能尚不足以充分刻画更高灵巧度操纵任务中的细微差别，从而导致 policy 优化效率受限。未来的研究方向包括：将强化学习扩展到 auto-regressive VLA 之外，训练基于 diffusion 的 policy [7,68]；以及利用大规模真实世界经验，进一步探索在线自我改进能力。

7 参考文献

参考文献条目较多，此处保留原论文英文文献列表，请参阅原文 PDF 的 References 一节。